

Rapport de projet — Système intelligent de prédiction du churn client

Module : M1 Dev. Manager Full Stack — Data Science (EFREI, 2025-26)

Sujet 2 : Rétention client et évaluation du risque de revenus

Tâche retenue : prédiction du churn (classification binaire)

Auteur(s) : [À compléter — noms du groupe] · **Date :** 26 juin 2026

1. Contexte et objectif

Les entreprises par abonnement (SaaS, télécom, e-commerce) perdent une partie de leurs clients chaque mois (*churn*). Or retenir un client coûte nettement moins cher que d'en acquérir un nouveau. L'objectif de ce projet est de construire un **système d'aide à la décision** capable de :

1. **prédire** la probabilité qu'un client résilie son abonnement,
2. **expliquer** les facteurs de ce risque,
3. **mettre cette prédiction à disposition** d'un utilisateur métier (responsable marketing / CRM) via un dashboard interactif et une API REST.

La valeur métier visée : permettre de **cibler les actions de rétention** sur les clients réellement à risque, avant leur départ.

2. Données et analyse exploratoire (EDA)

Jeu de données : `customer_churn.csv` — **10 000 clients**, 32 colonnes (30 variables explicatives : 19 numériques, 11 catégorielles ; 1 identifiant ; 1 cible `churn`).

2.1 Qualité des données

- **Valeurs manquantes** : une seule variable concernée, `complaint_type` (2 045 NaN, soit 20,5 %).
Interprétation métier : absence de réclamation → traitée comme une modalité explicite `"None"` plutôt que supprimée.
- **Identifiant** `customer_id` : unique, sans information prédictive → retiré.
- Aucun doublon, types cohérents, échelles plausibles (NPS $\in [-100, 100]$, CSAT $\in [1, 5]$, taux $\in [0, 1]$).

2.2 Variable cible : un fort déséquilibre

La classe positive (`churn = 1`) représente **10,2 %** des clients (1 021 churners / 8 979 fidèles), soit un **ratio de déséquilibre de 8,8:1**. Conséquence méthodologique majeure : l'*accuracy* est trompeuse (un modèle prédisant « aucun churn » atteindrait ~90 % d'*accuracy* tout en étant inutile). Les métriques retenues seront donc le **Recall**, le **F1-score** et surtout la **PR-AUC** (aire sous la courbe précision-rappel), plus informative que la ROC-AUC en contexte déséquilibré.

2.3 Premiers signaux métier

L'analyse croisée du taux de churn par variable révèle :

Variable	Lecture
<code>payment_failures</code>	≤ 1 échec $\rightarrow \sim 8,8\%$ de churn ; ≥ 2 échecs $\rightarrow 21\%$ à 33% . Signal très fort.
<code>csat_score</code> (satisfaction)	corrélation marquée avec le churn (cf. §5).
<code>contract_type</code> , <code>customer_segment</code> , <code>price_increase_last_3m</code>	taux quasi constants ($\sim 10\%$) $\rightarrow$ faible pouvoir discriminant.

Ce dernier point est important : plusieurs variables *a priori* intuitives portent peu de signal. L'EDA évite ainsi de sur-interpréter des variables non déterminantes.

3. Méthodologie

3.1 Pipeline de préparation (anti-fuite de données)

Le préprocessing est encapsulé dans un `Pipeline` scikit-learn, **appris uniquement sur le jeu d'entraînement** (y compris à l'intérieur de chaque pli de validation croisée), garantissant l'absence de fuite vers le test : - **Numériques** : imputation médiane $\rightarrow$ standardisation (`StandardScaler`) ; - **Catégorielles** : imputation constante (`"None"`) $\rightarrow$ encodage one-hot (`handle_unknown="ignore"` pour gérer toute modalité inconnue en production).

3.2 Protocole d'évaluation

- **Découpage** train/test stratifié 80/20 (préserve le ratio de churn).
- **Validation croisée stratifiée** (5 plis) pour la sélection de modèle.
- **Métrique de sélection** : PR-AUC.
- Graine aléatoire fixée (`seed = 42`) $\rightarrow$ reproductibilité.

3.3 Gestion du déséquilibre

Quatre stratégies comparées sur une régression logistique de référence : absence de rééquilibrage (`class_weight="balanced"`), **SMOTE**, sur-échantillonnage aléatoire, sous-échantillonnage aléatoire. Le rééchantillonnage est appliqué **à l'intérieur de la CV** (sur les plis d'entraînement uniquement) via un pipeline `imbalanced-learn`, ce qui évite toute fuite.

3.4 Modèles comparés

Quatre familles, du plus simple au plus complexe :

Modèle	Rôle
Régression logistique	baseline interprétable
Random Forest	ensemble d'arbres, non-linéaire
HistGradientBoosting	gradient boosting
MLP (réseau de neurones)	Deep Learning (exigence du sujet)

3.5 Optimisation du seuil de décision

Le seuil par défaut de 0,5 n'est pas adapté à un problème déséquilibré. Le seuil optimal est recherché sur les **probabilités out-of-fold** du train (pour éviter le sur-ajustement) en maximisant le F1-score.

4. Résultats

4.1 Étude du déséquilibre (baseline LogReg, PR-AUC en CV)

Stratégie	PR-AUC	ROC-AUC
Aucune (<code>class_weight</code>)	0,227	0,717
Sur-échantillonnage	0,226	0,717
SMOTE	0,226	0,715
Sous-échantillonnage	0,218	0,710

Lecture : les techniques de rééchantillonnage **n'améliorent pas** la PR-AUC par rapport à la pondération des classes ; le sous-échantillonnage la dégrade même légèrement (perte d'information). Le rééquilibrage joue ici surtout sur le **point de fonctionnement** (seuil), pas sur le pouvoir de ranking du modèle. SMOTE est conservé pour la suite afin d'illustrer une démarche complète, mais `class_weight` serait un choix tout aussi défendable et plus simple.

4.2 Comparaison des modèles (CV stratifiée, rééquilibrage SMOTE)

Modèle	PR-AUC	ROC-AUC
Random Forest ★	0,267	0,794
HistGradientBoosting	0,259	0,789
Régression logistique	0,226	0,715
MLP (Deep Learning)	0,167	0,631

(PR-AUC d'un classifieur aléatoire = taux de churn = 0,102 ; le Random Forest fait donc ~2,6× mieux que le hasard.)

Analyse critique du Deep Learning. Le MLP est **le modèle le moins performant**, nettement derrière les modèles à arbres. Ce résultat illustre l'un des objectifs pédagogiques du sujet : *le Deep Learning n'est pas toujours supérieur*. Sur des **données tabulaires hétérogènes** de taille modérée, les ensembles d'arbres (Random Forest, Gradient Boosting) capturent mieux les interactions et sont plus robustes, tandis qu'un réseau de neurones nécessite davantage de données et un réglage plus fin pour rivaliser — pour un coût de calcul supérieur. Le compromis performance/complexité penche donc clairement vers le Random Forest.

4.3 Modèle final et impact du seuil

Modèle retenu : Random Forest (`max_depth=None` , `min_samples_leaf=1`). Seuil de décision optimisé : **0,19**.

Performance sur le **jeu de test** :

	Seuil 0,50	Seuil 0,19 (optimisé)
Recall (churners détectés)	0,00	0,78
Precision	0,00	0,24
F1-score	0,00	0,37
Accuracy	0,895	0,724
PR-AUC / ROC-AUC	0,262 / 0,793	0,262 / 0,793

Résultat clé : au seuil par défaut de 0,5, le modèle **ne détecte aucun churner** (recall = 0) malgré une accuracy de 90 % — exactement le piège du déséquilibre. L'abaissement du seuil à 0,19 permet de détecter **78 % des churners**, au prix d'une précision plus faible (24 %). Ce compromis est **pertinent en contexte métier** : un faux négatif (churner non détecté) coûte un client perdu, alors qu'un faux positif ne coûte qu'une sollicitation marketing. Le réglage du seuil traduit donc une décision business, pas seulement statistique.

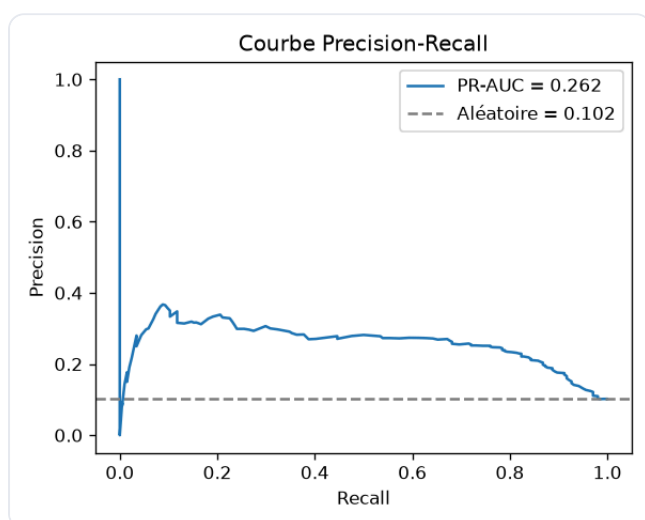


Figure 1 — Courbe Precision-Recall du modèle final (test).

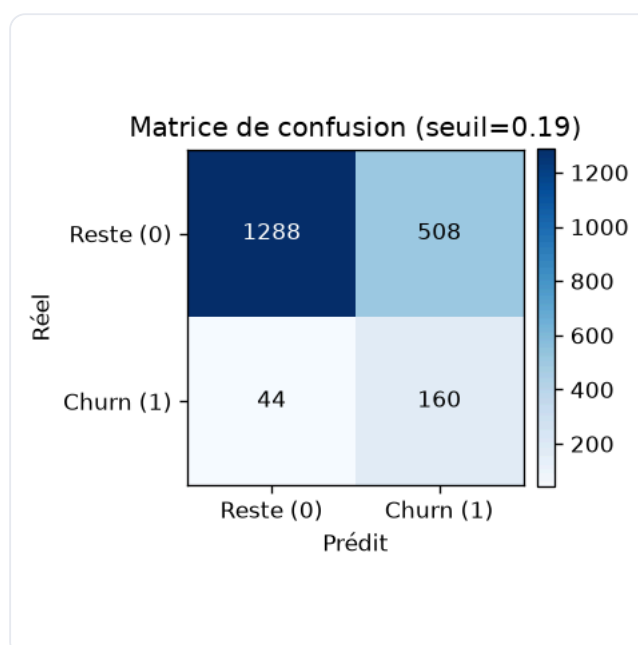


Figure 2 — Matrice de confusion au seuil optimisé (0,19).

5. Interprétabilité

L'importance des variables a été analysée par **importance native du Random Forest**, **importance par permutation** (agnostique au modèle) et **SHAP** (explications locale et globale).

Principaux facteurs de churn :

Rang	Variable	Lecture métier
1	csat_score	satisfaction client — facteur dominant et de loin
2	payment_failures	échecs de paiement — risque qui triple dès 2 échecs
3	tenure_months	ancienneté — les clients récents sont plus volatils
4-5	monthly_logins , total_revenue	engagement et valeur du client
6+	survey_response , payment_method	satisfaction déclarée, moyen de paiement

Ces facteurs sont **cohérents avec la logique métier** : un client peu satisfait, rencontrant des incidents de paiement et faiblement engagé est un candidat naturel au départ. SHAP permet en outre d'expliquer une prédiction **individuelle** (« ce client est à risque à 78 % principalement à cause d'un CSAT faible et de paiements ratés »), ce qui rend le modèle exploitable par un opérationnel.

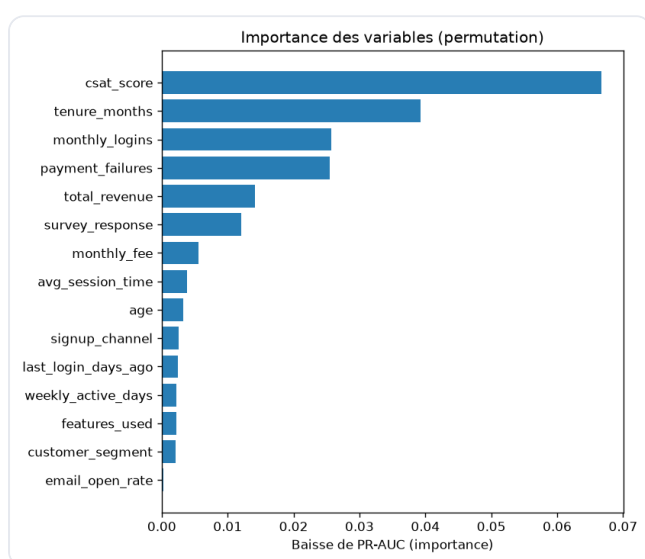


Figure 3 — Importance des variables par permutation.



Figure 4 — Résumé SHAP (impact des variables sur la prédiction).

6. Limites et biais

- **Pouvoir prédictif modéré.** Une PR-AUC de 0,26 (vs 0,10 aléatoire) et une ROC-AUC de 0,79 indiquent un signal réel mais limité. Le jeu de données étant **synthétique**, une partie du comportement de churn n'est probablement pas capturée par les variables disponibles.
- **Précision faible au seuil retenu (24 %)** : trois clients ciblés sur quatre ne churneraient pas. Acceptable si le coût d'une action de rétention est faible, à réévaluer sinon.
- **Variables peu informatives** (contrat, segment, hausse de prix) : leur faible signal pourrait refléter une limite des données plutôt qu'une réalité métier.
- **Stabilité du Deep Learning** : le MLP, sensible à l'initialisation et au réglage, pourrait être amélioré, mais resterait coûteux face aux arbres.
- **Dérive temporelle** : le modèle est statique ; en production, les comportements évoluent et un réentraînement périodique serait nécessaire.

7. Recommandations actionnables

Pour le métier (rétention) : 1. **Prioriser les clients à ≥ 2 échecs de paiement** : leur taux de churn (21-33 %) est 2 à 3 fois la moyenne. Action : fiabiliser le paiement (relance, mise à jour du moyen de paiement, échéancier). 2. **Surveiller la satisfaction (CSAT/NPS)** comme indicateur avancé n°1 : déclencher un contact proactif sous un certain seuil. 3. **Sécuriser les clients récents** (faible ancienneté) par un onboarding renforcé.

Pour la solution technique : 4. Enrichir les données par du **feature engineering** (ex. ratio tickets/ancienneté, tendance d'usage) pour gagner en pouvoir prédictif. 5. Adopter `class_weight` plutôt que SMOTE (performances équivalentes, plus simple). 6. Mettre en place un **suivi de dérive** et un réentraînement périodique.

8. Industrialisation et reproductibilité

La solution dépasse le notebook : elle est structurée comme un mini-produit logiciel.

- **API REST (FastAPI)** : endpoints `POST /predict` (probabilité + classe + niveau de risque), `GET /health`, `GET /model-info`, avec validation des entrées (Pydantic) et gestion d'erreurs (HTTP 422/503). Le pipeline de préprocessing est sérialisé avec le modèle pour garantir une transformation identique entre entraînement et inférence.
 - **Dashboard décisionnel (Streamlit)** : KPIs, scoring d'un client, **simulation de scénario de rétention**, explication des facteurs, comparaison des modèles. Le dashboard **consomme l'API** (architecture réaliste Front / API / Modèle).
 - **Reproductibilité** : code modulaire (`src/churn/`), `Makefile` (`setup / train / api / dashboard / test`), `requirements.txt`, graine fixe, artefacts et métriques versionnés, tests `pytest` (préprocessing + API).
-

9. Conclusion

Le projet livre un système complet et défendable : d'un dataset brut à une solution explicable et industrialisée. La démarche met l'accent sur la **rigueur méthodologique** — gestion explicite du déséquilibre, absence de fuite de données, optimisation du seuil, comparaison multi-modèles — plutôt que sur la seule maximisation d'un score. Deux enseignements se dégagent : (1) sur des données tabulaires, **un modèle simple (Random Forest) surpasse le Deep Learning** pour un coût bien moindre ; (2) en contexte déséquilibré, **le réglage du seuil de décision est aussi déterminant que le choix du modèle**. Le système est prêt à être utilisé par un profil métier et extensible (feature engineering, suivi de dérive) pour gagner en performance.